

PRD: VibeKit

Version: 1.3 | **Status:** Ready

1. Product Summary

VibeKit is a web-based, highly extendable personal developer dashboard built to host custom local utilities and productivity tools, launching with an MVP of a single, client-side HTML Beautifier & Exporter tool.

2. Tech Stack

| **Layer** | **Technology** |

| **Frontend** | Next.js 14 (App Router), React, Tailwind CSS |

| **Backend** | — (not needed in MVP) |

| **UI Components** | shadcn/ui (Radix Primitives) + Lucide Icons |

| **Package Manager** | npm (or pnpm) |

| **Deployment** | Vercel |

| **Database** | — (not needed in MVP) |

3. Core User Flows

Flow 1: Tool Selection & Access

User navigates to the dashboard home page / → User views a grid list of available utilities → User selects the "HTML Beautifier & Exporter" card → Dashboard interface transitions seamlessly to the HTML Beautifier workspace.

Flow 2: HTML Beautification

User pastes unformatted, minified, or disorganized HTML into the raw input panel → User clicks the "Beautify" button → The client-side logic processes the input, formats it with standard 2-space indentation, and renders the formatted output in a read-only syntax-highlighted editor view.

Flow 3: Copy and Download Export

User views the newly beautified code → User clicks "Copy Code" → System copies raw formatted code to clipboard and displays a success toast → User clicks "Download .html" → System generates a client-side blob and triggers a browser file download of beautified.html.

4. Pages & Components

1. Dashboard Landing Page (/)

- **Purpose:** A clean hub displaying all available tools, structured to easily support new tools in the future.
- **Key Data Displayed:** Grid of tool cards with titles, descriptions, categories, and status tags (e.g., "Active", "Beta", "Coming Soon").
- **Actions Available:** Click on an active tool card to open/route to it; search/filter tools by name or tag.
- **Main Components:**
 - SidebarNavigation: Persistent collapsible navigation to swap between dashboard, settings, and tools.
 - ToolCardGrid: Responsive CSS Grid rendering tool configurations.
 - SearchBar: Basic filtering input to quickly find tools.

2. HTML Beautifier Workspace (/tools/html-beautifier)

- **Purpose:** High-performance, client-side code formatting utility.
- **Key Data Displayed:** Raw input textarea, formatted output viewport, and code analytics (e.g., character count, lines saved).
- **Actions Available:** Paste raw HTML, click "Beautify", copy output to clipboard, download output as a .html file, or clear the editor.
- **Main Components:**
 - WorkspaceLayout: Side-by-side (on desktop) or stacked (on mobile) dual-editor layout.
 - RawInputArea: Standard textarea styled to match code editors, supporting native tab indentation.
 - BeautifiedOutputArea: Read-only code canvas featuring basic token syntax highlighting (or standard monospace output styling).
 - ControlBar: Header actions containing buttons for "Beautify", "Copy", "Download", and "Reset".

5. Data Model

No database in MVP. All processing is client-side or via stateless API routes.

6. Acceptance Criteria

Feature 1: Landing Hub and Navigation

- Given a user lands on /, when they view the page, then they should see a highly professional grid with "HTML Beautifier" listed as active, and placeholder cards for future

tools (e.g., "AI Agent Interface" marked as "Coming Soon").

- Given a user clicks the "HTML Beautifier" card, when clicked, then they must be routed instantly to /tools/html-beautifier without layout shifting.

Feature 2: HTML Beautification Engine

- Given the raw input text area is empty, when the user clicks "Beautify", then the app should display an error toast warning "Please enter HTML to beautify" and prevent execution.
- Given a user pastes messy, valid HTML, when they click "Beautify", then the system must format it locally with consistent 2-space nesting, tags closed correctly, and render it in the output pane within 100ms.
- Given a user inputs broken/invalid HTML, when they click "Beautify", then the system should handle parsing errors gracefully, notifying the user via a friendly error state rather than breaking the UI.

Feature 3: Action Operations (Copy & Export)

- Given there is beautified output, when the user clicks the "Copy" button, then the system must write the formatted string to the user's system clipboard and trigger a "Copied to clipboard!" toast notification.
- Given there is beautified output, when the user clicks "Download .html", then the system must automatically initiate a native .html file download named beautified-[timestamp].html without server-side processing.

7. Out of Scope (MVP)

- User account registration, login, or session state syncing.
- Persistent database history storing previous HTML transformations.
- Custom user configuration profiles (e.g., custom indentation rules like tabs vs spaces).
- Full-featured code IDE integrations (like Monaco Editor). Only a lightweight, styled textarea or light code box is planned.
- Real-time AI integrations or other planned agent tools.

8. UI/UX

The UI and design system will be following based on the 'uiux' project folder.

9. Optional

- **Background:** This dashboard serves as a highly scalable playground built specifically using AI-assisted coding methodologies. The goal is to establish a solid scaffolding where additional micro-tools, helper utilities, and LLM-assisted background tasks can be appended cleanly without codebase bloat.